

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA

Sensor Systems

Laboratory 04
Timers

02.10.2026 / 05.10.2026 | Federica Villa, Arianna De Vecchi

Contents

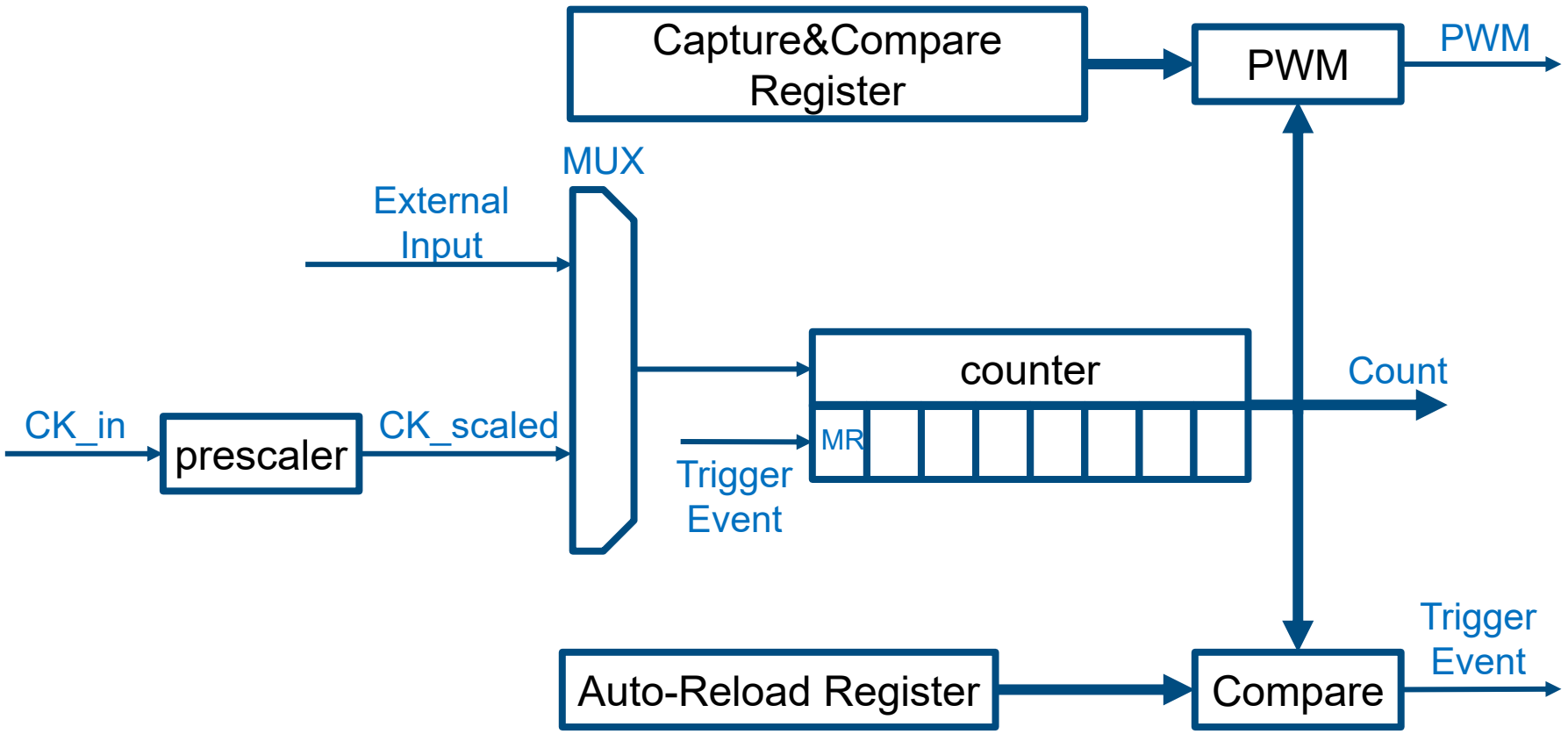
1. Timers

2. Functions and exercises

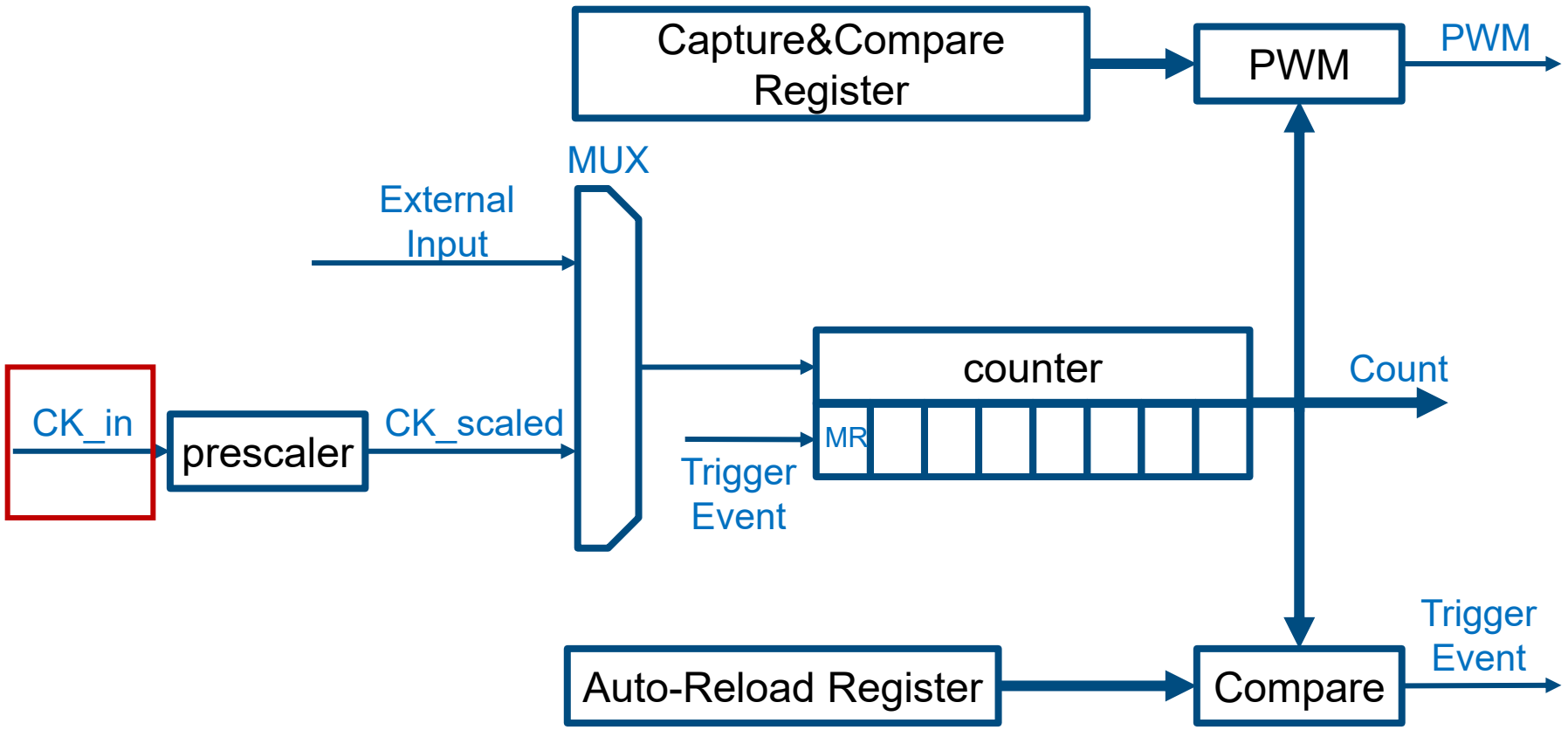
1. Timers

01

Timer block diagram



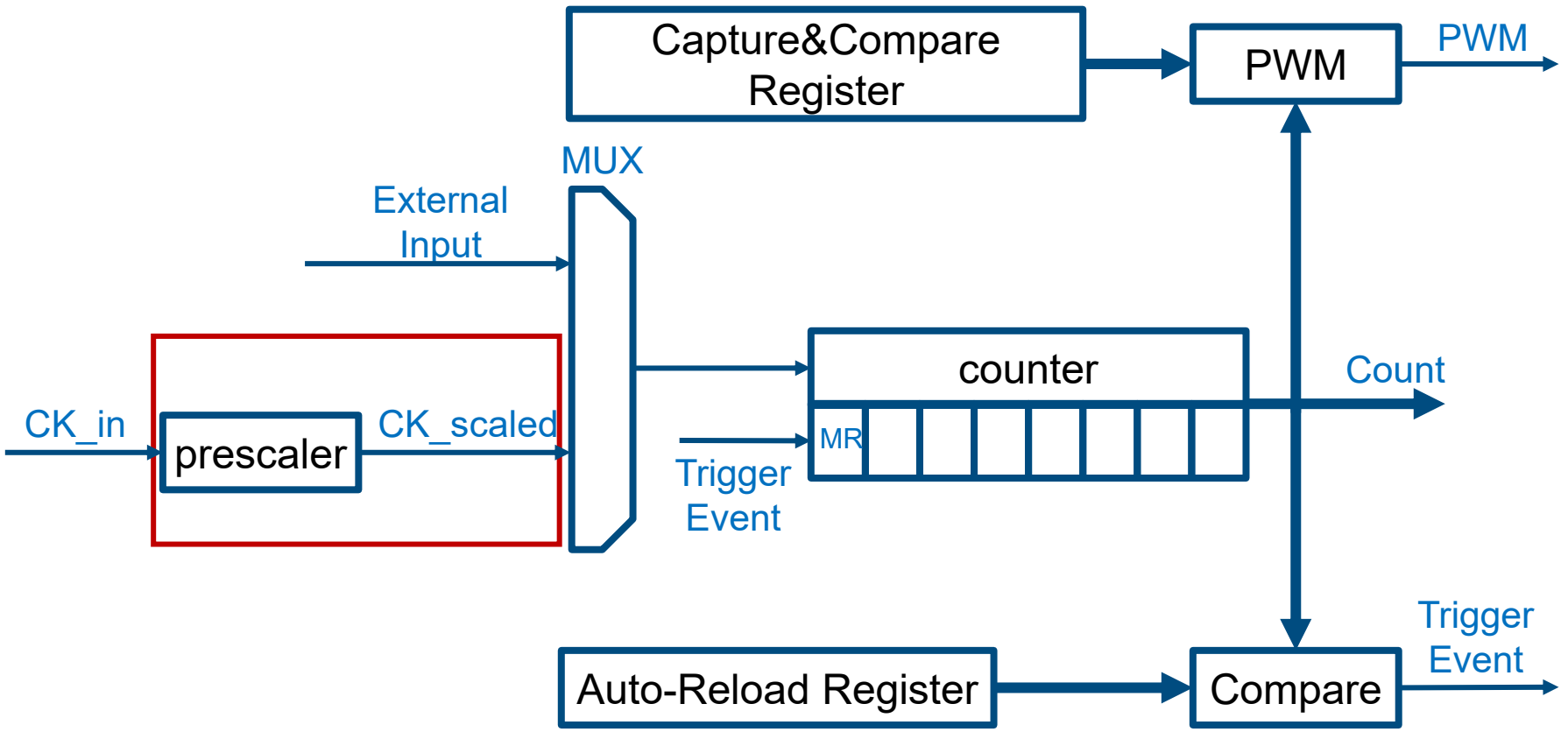
Timer block diagram



Input clock (CK_in)

- The timer is a counter that needs a steady heartbeat to advance
- That heartbeat is the input clock, CK_in, coming from the chip

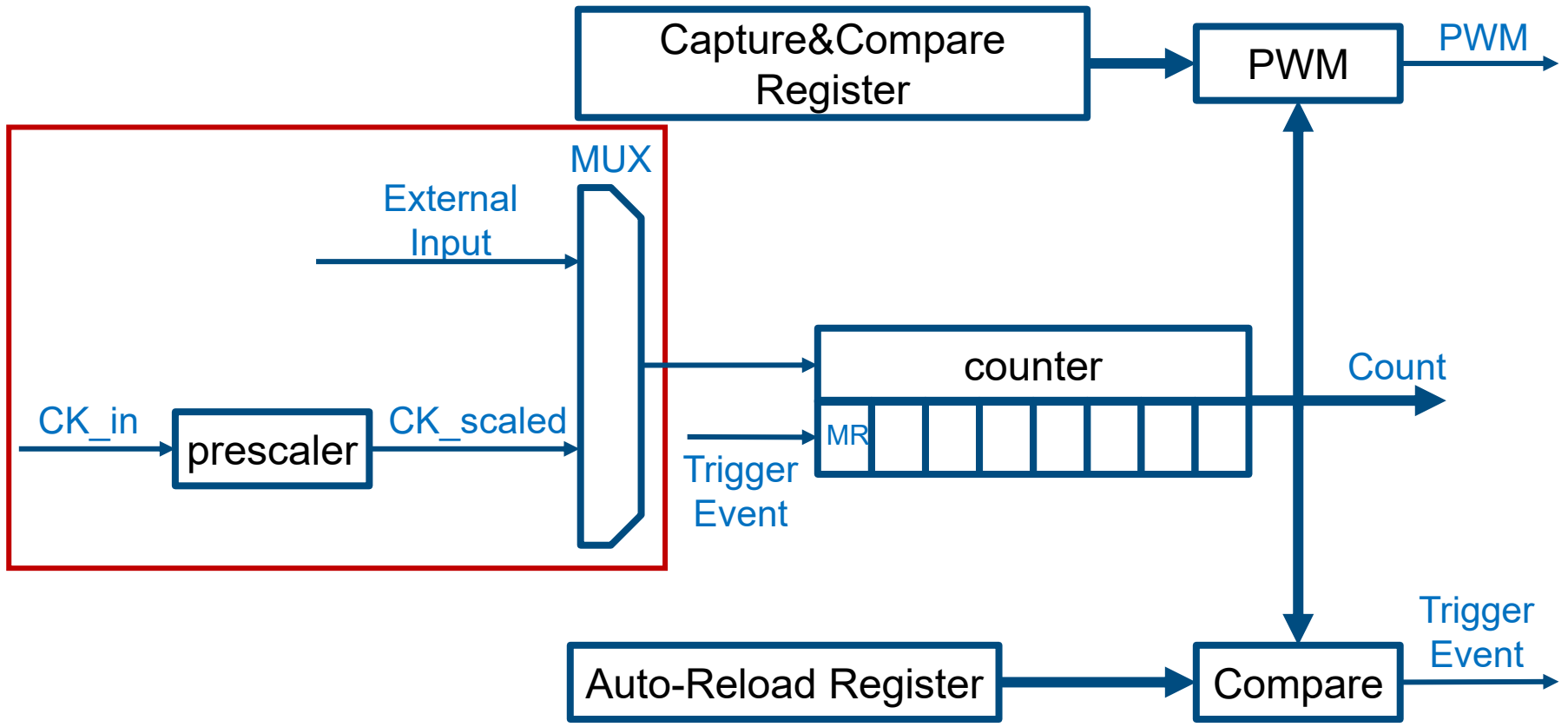
Timer block diagram



Prescaler → CK_scaled

- The chip clock is usually too fast for the counter
- The prescaler divides it by a chosen factor (e.g. ÷1000)
- Output is a slower tick, CK_scaled → this sets how fast the counter counts

Timer block diagram

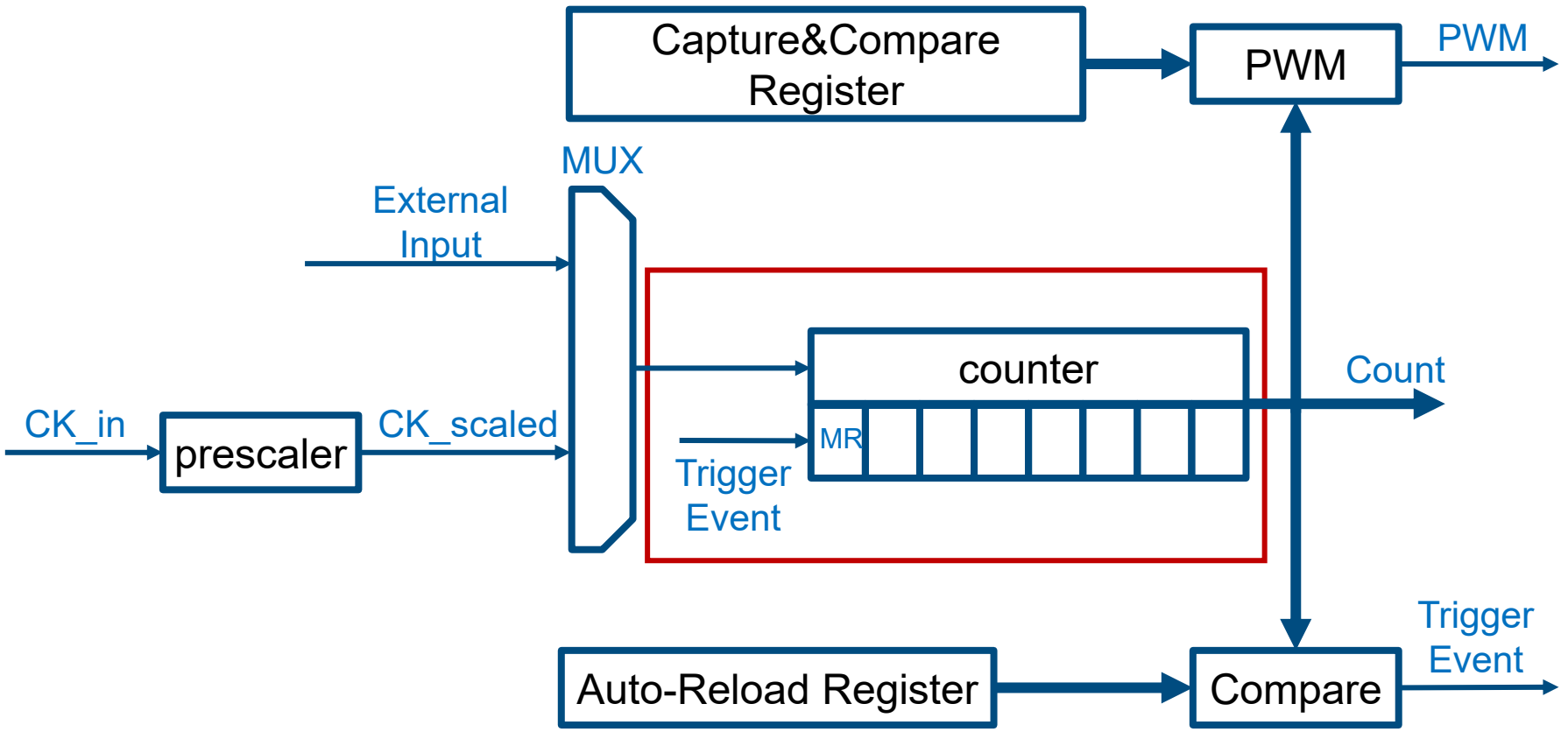


External input (via the MUX)

A MUX selects what actually drives the counter:

- **CK_scaled** → the counter measures time (internal clock)
- **External Input** → the counter counts external pulses, not time
- **Same timer, two uses**: timebase or event/pulse counter

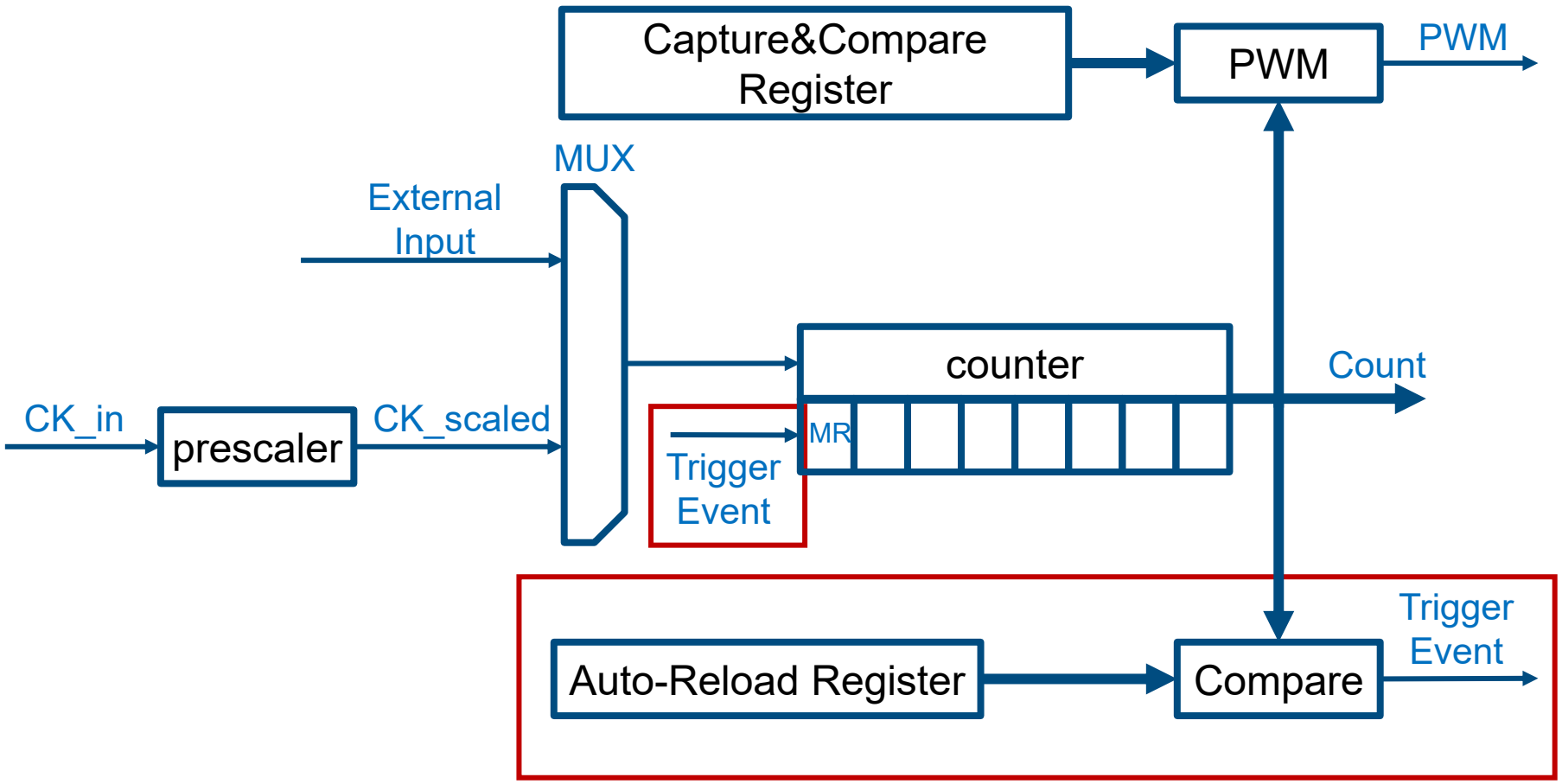
Timer block diagram



Counter

- Driven by the tick chosen in the MUX; each tick adds 1 → the current value is the Count
- **MR** = the reset input: it brings the count back to 0
- Counter can be set in Up, Down, Up/Down mode.

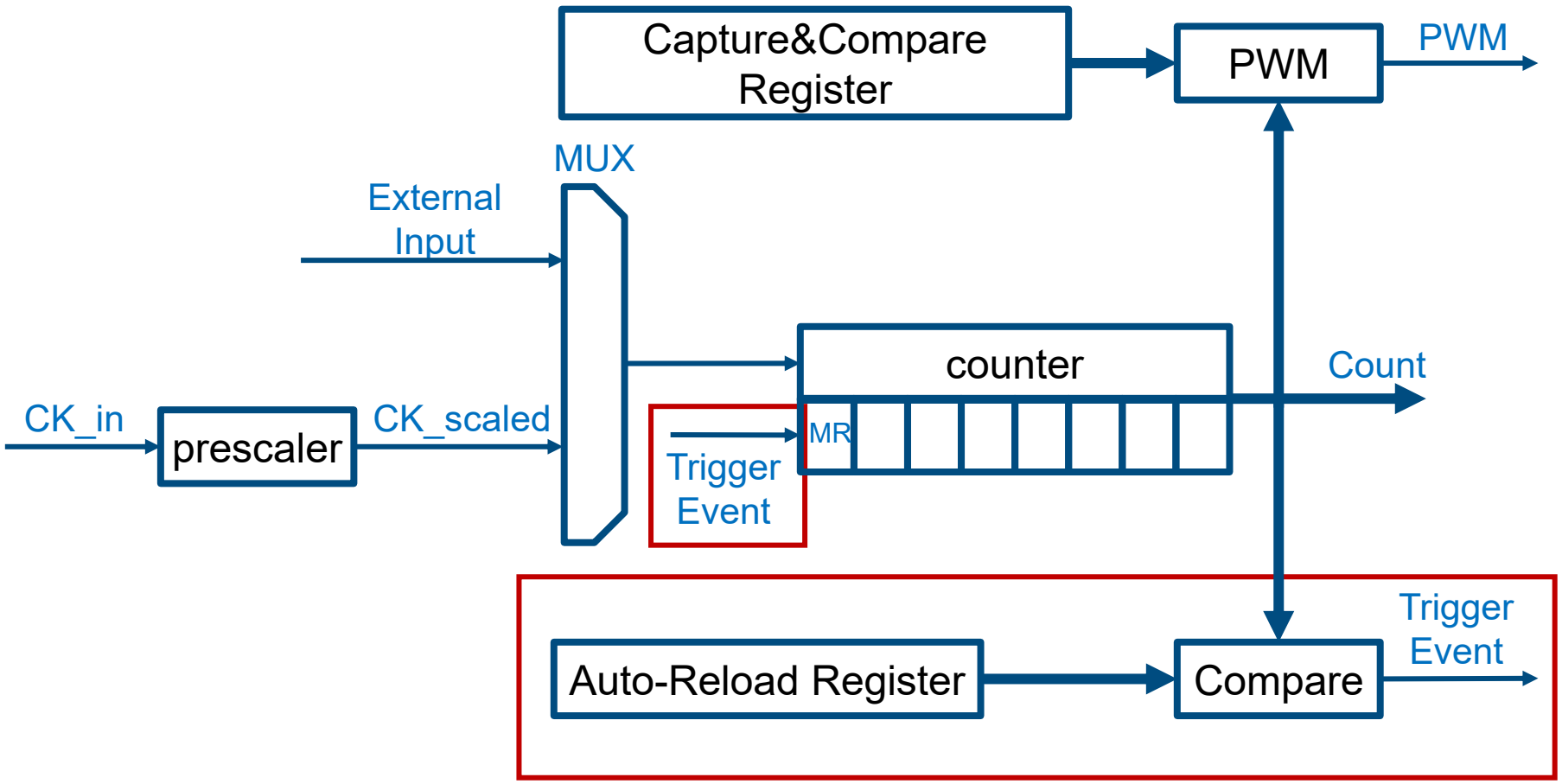
Timer block diagram



Auto-Reload Register:

- Holds the number the counter is allowed to reach
- The Compare block checks the count against it at every tick

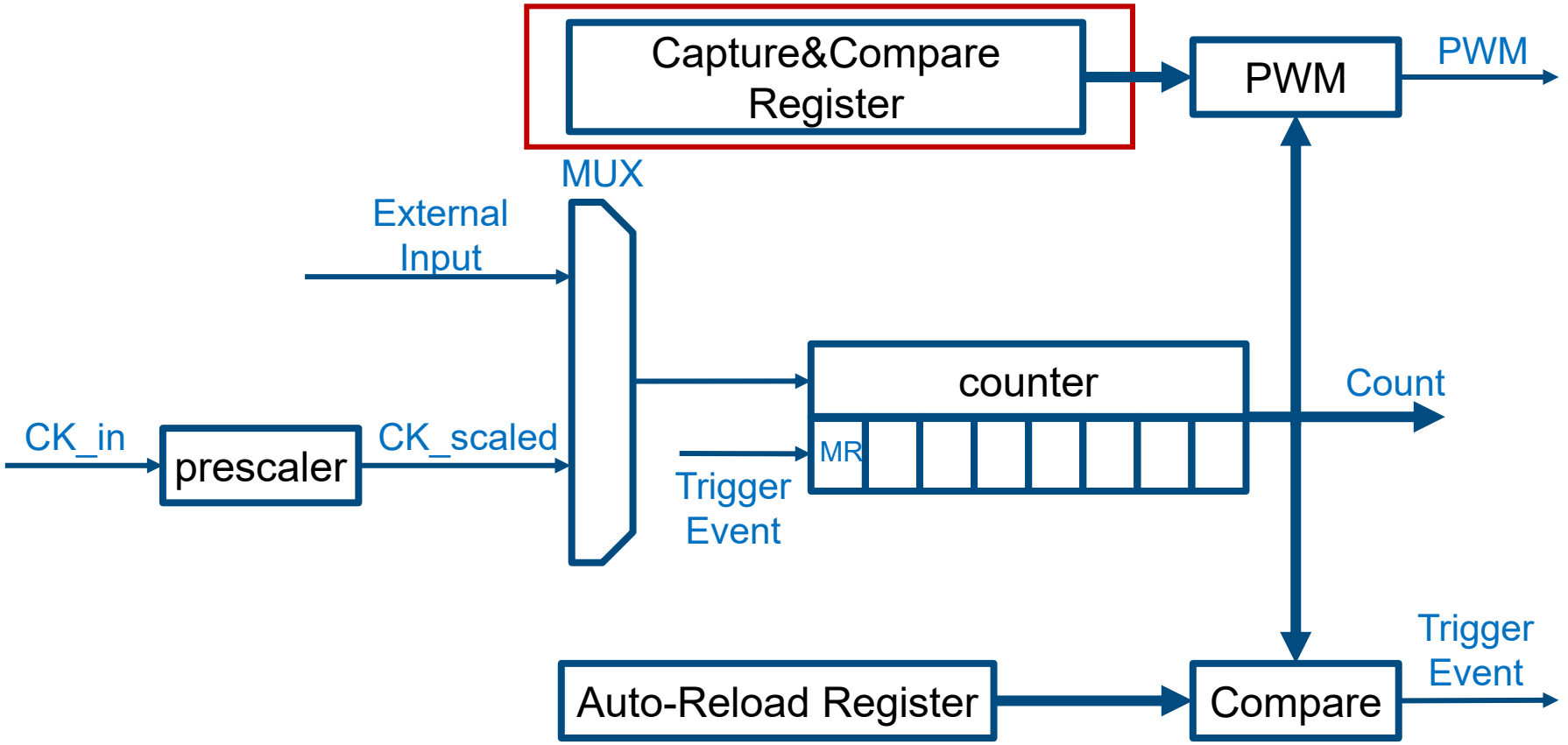
Timer block diagram



Trigger Event

- When count = Auto-Reload value, Compare fires a Trigger Event
- That event resets the counter (via MR) → it starts again from 0
- So the counter runs in a loop; the length of the loop = the period

Timer block diagram

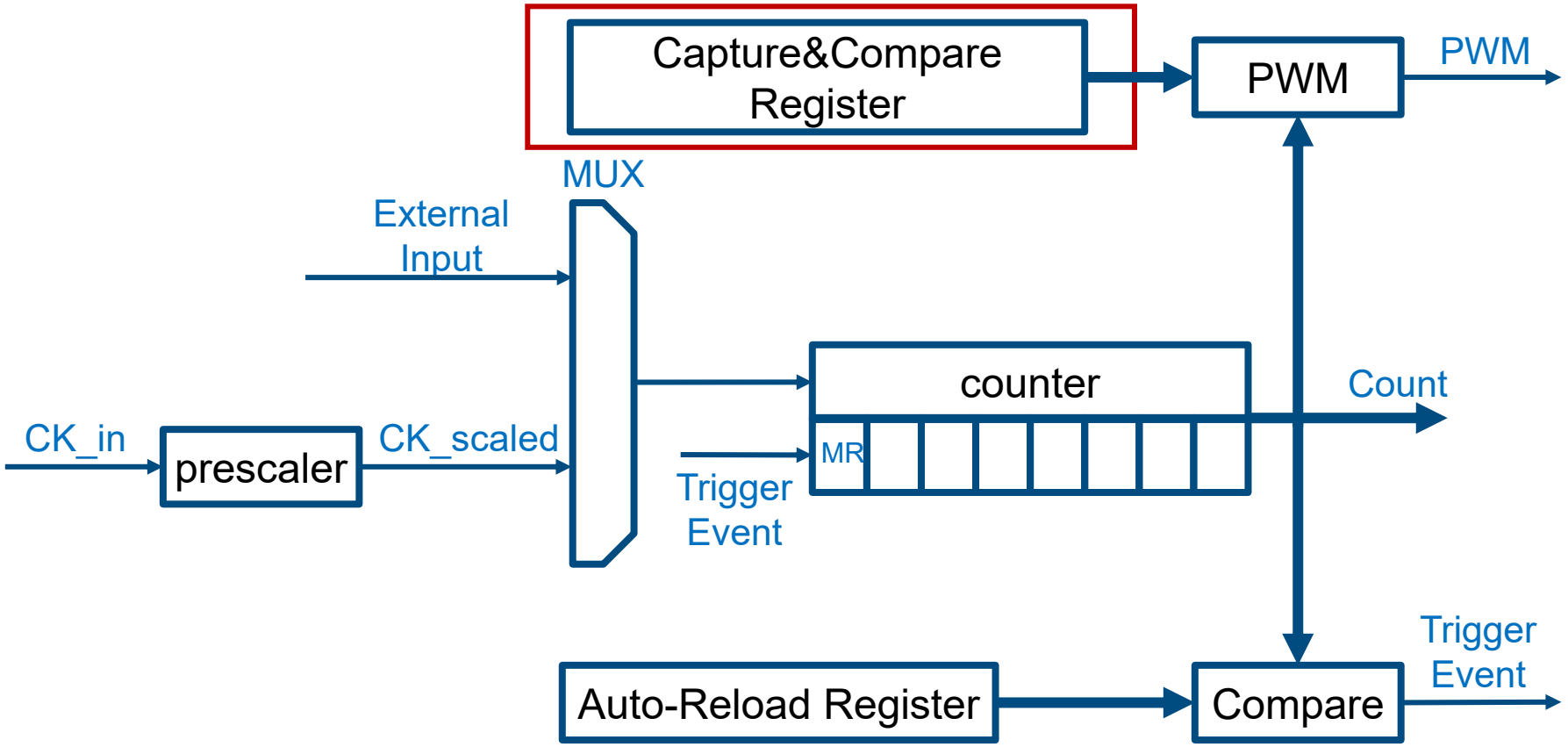


Capture & Compare Register

It is a threshold inside the count range

- Holds a value between 0 and the Auto-Reload value

Timer block diagram



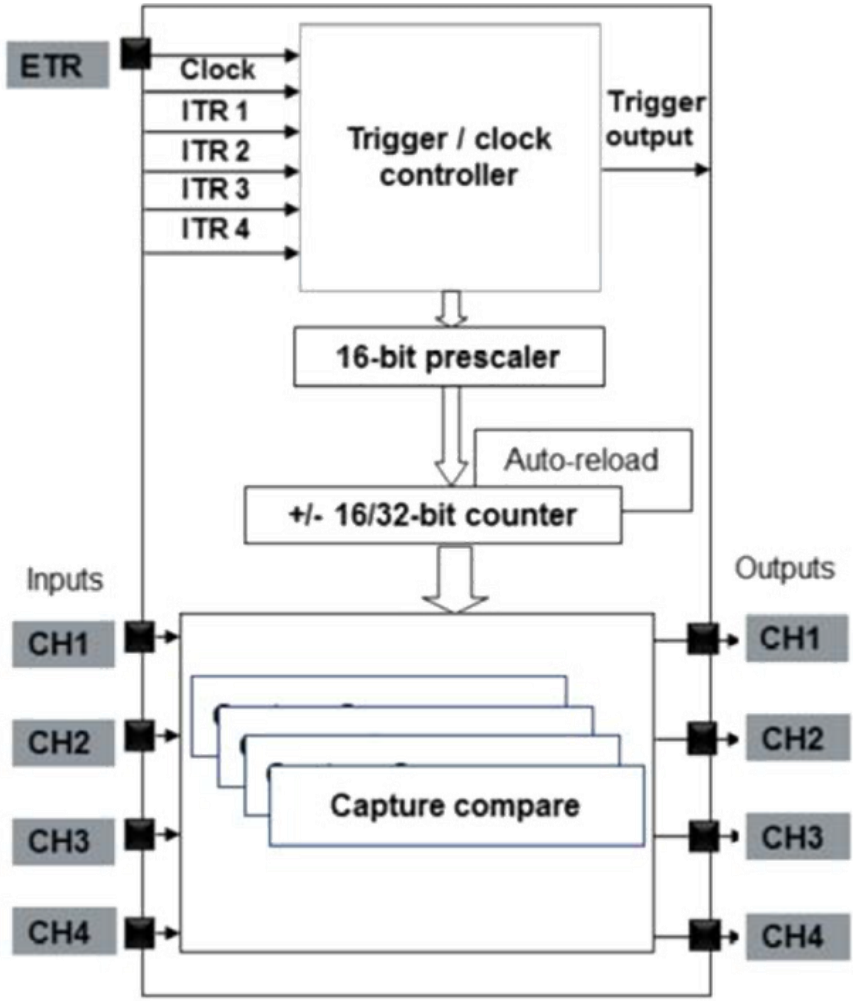
The **PWM** block compares the count to the Capture&Compare register:

- Output is high while count is below the threshold, low above it
- The counter ramps up and resets every period, so the output becomes a repeating square wave = PWM

Period / frequency ← prescaler + Auto-Reload

Duty cycle ← Capture & Compare

Timer structure



Trigger / clock controller:

- selects input clock
- synchronization with other clocks
  (Master → Output Trigger
  Slave → Input Trigger)

Prescaler

Divides the frequency the counter increases, skipping a portion of the incoming events.

Counter

Up, Down, Up/Down counter

Capture and Compare registers

One for each channel

Auto-reload register

Sets the timer FSR

Timer functionalities

Time base

Generates triggers and interrupts at the overflow (or at the ARR value)
(e.g., trigger ADC conversion, software management, generic interrupts).

Input capture

Capture the counter value in the capture register. This allows the timer to record the time at which an externa event occurs.

Output compare

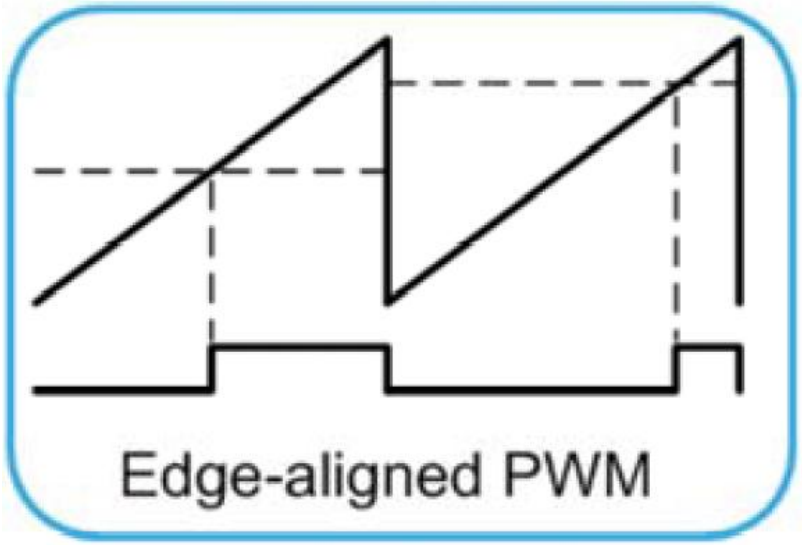
Every time that the compare value is reached:

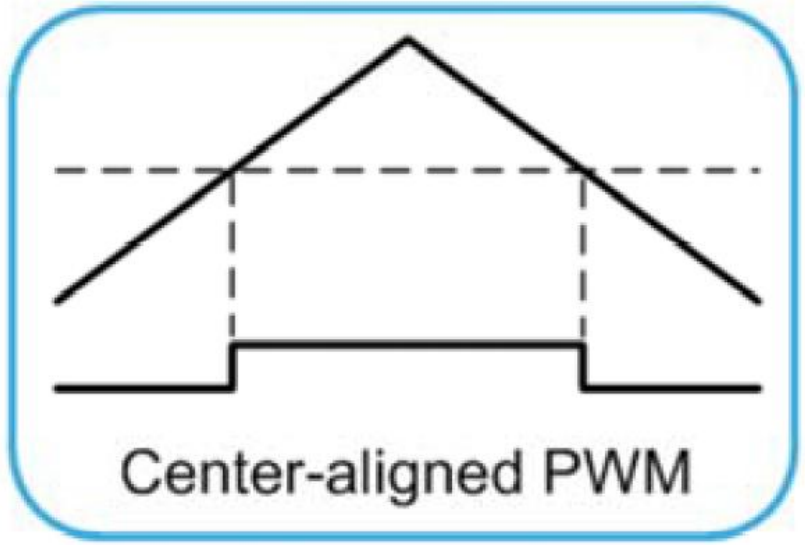
- corresponding output pin is SET/RESET/TOGGLE/UNCHANGED
- generate an interrupt / trigger

PWM generation

- Edge aligned: counter Up or Down mode
- Center aligned: counter Up/Down mode

Pulse Width Modulator (PWM)

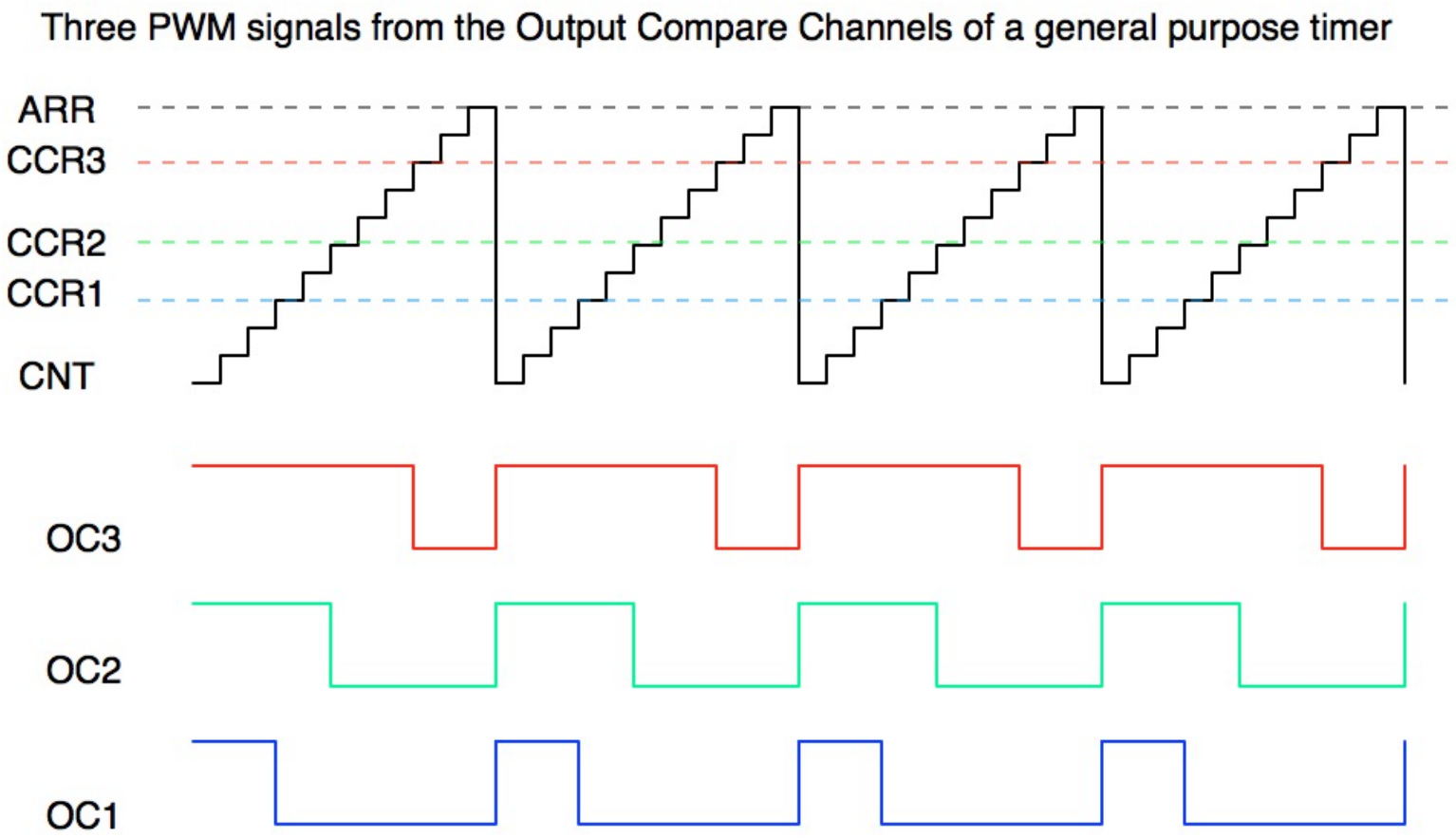




PWM frequency: $f_{PWM} = \frac{f_{TIM}}{(ARR+1)\cdot(PSC+1)}$

PWM Duty Cycle: $DC = \frac{CCRx+1}{ARR+1}$

PWM generation



Registers

Prescaler (PSC)

Note that in order to divide by N the clock frequency you need to write N-1 in the Prescaler register

Auto Reload Register (ARR) → named "**period**" in the CubeMX
FSR of the counter (counter overflow)

Capture / Compare Register (one for each channel) **(CCRx)** → named "**pulse**" in the CubeMX
Capture = counter value when trigger is received
Compare = counter value which generate an event (trigger or PWM commutation).

A complete list of the timer registers can be found in the Reference Manual.

2. Functions and exercises

02

Timer HAL functions

To start (stop) the TIM Base generation (x_IT with interrupt generation)

HAL_StatusTypeDef **HAL_TIM_Base_Start**(TIM_HandleTypeDef * htim)

HAL_StatusTypeDef **HAL_TIM_Base_Start_IT**(TIM_HandleTypeDef * htim)

HAL_StatusTypeDef **HAL_TIM_Base_Stop**(TIM_HandleTypeDef * htim)

HAL_StatusTypeDef **HAL_TIM_Base_Stop_IT**(TIM_HandleTypeDef * htim)

To start (stop) the PWM signal generation

HAL_StatusTypeDef **HAL_TIM_PWM_Start**(TIM_HandleTypeDef *htim, uint32_t Channel)

HAL_StatusTypeDef **HAL_TIM_PWM_Stop**(TIM_HandleTypeDef *htim, uint32_t Channel)

Timer callbacks (remember to specify the code for each timer as for GPIOs)

void **HAL_TIM_PeriodElapsedCallback**(TIM_HandleTypeDef *htim)

Project 2a: Blinking LED (PWM)

Objective of this project is to **blink** the NUCLEO board **green LED** at **1 Hz** with **50% DC**, using a **PWM**.

Project 2a hints

- Use the NUCLEO board Manual to find the GPIO connected to the green LED.
- Set in CUBE the GPIO of the green LED as the output of a PWM
- Check the TIMER clock frequency and configure the corresponding timer in order to obtain the required frequency and duty cycle.
- Generate the c code.
- In the main, start the correct channel of the timer connected to the LED.
- Debug and verify the blinking frequency.

Project 2b: Play note through speaker

Objective of this project is to **play a tone** for **3 seconds** using the **speaker**, using a **PWM**.

Timer configuration for the various notes can be computed using the file "Musical notes.xslx" available on WeBeep

Project 2b hints

- Use the NUCLEO board Manual to find the GPIO connected to the speaker.
- Set in CUBE the GPIO of the speaker as the output of a PWM
- Check the TIMER clock frequency and configure the corresponding timer in order to obtain a 440 Hz tone (LA4 / A4 note) with 50 % duty cycle.
- Generate the c code.
- In the main, start the correct channel of the timer connected to the speaker. Stop it a few seconds later.
- Debug and verify the note frequency. (Spectrum analyzer apps are easily available for smartphones)

HOMEWORK 3

Objective of this project is to **play a song** using the **speaker** when the **microphone** detects a loud **sound.**

Try to use a **timer interrupt** to move through the notes and not the HAL_Delay function.

HOMEWORK 3



Timer configuration for the various notes can be computed using the file "Musical notes.xslx" available on WeBeep

HOMEWORK 3

Hint: start from playing a song once, without the microphone interrupt.

- Start from the same configuration as in the previous project.
- Identify the setup functions that initialize the timer frequency and pulse duration.
- Use the required code from those functions to create a new function to setup a new frequency and pulse duration of the timer.
- Define a struct to store the note frequency and duration
- Create an array of structs to store the song
- Play the song by stepping through the array, every time setting the correct frequency and for the correct duration of the note.

HOMEWORK 3

Now add the microphone interrupt!

- Start from the same configuration as in the previous project.
- Setup the correct interrupt and using the correct callback, play the song
- Is the song being played correctly? If not, try to debug your code and propose a possible solution to the problem.
- If the song is playing correctly, does it stop at the end? If not, debug your code and propose a possible solution to the problem.

HOMEWORK 3

Pin settings:

- EXTI8 – PA8 (D7): microphone, rising edge, starts the song
- TIM1 CH2 (PWM) – PA9 (D8): generates the note frequency for the buzzer
- TIM2 (update interrupt): measures note duration

HOMEWORK 3

Data structure:

```
struct note {
        int tone;     // TIM1 ARR value → pitch
        int duration;  // in units of TEMPO (75 ms)
};

struct note score[] = { {_SOL4, 6}, {_LA4, 2}, ...
```

HOMEWORK 3

```
//... and for the duration of the note (interrupt)
__HAL_TIM_SET_AUTORELOAD(&htim2,score[song_index].duration * TEMPO * 10);
```

$$f_{counter} = \frac{f_{timer}}{(PSC + 1)} = \frac{84.000.000}{8400} = 1\ kHz = 0{,}1\ ms$$

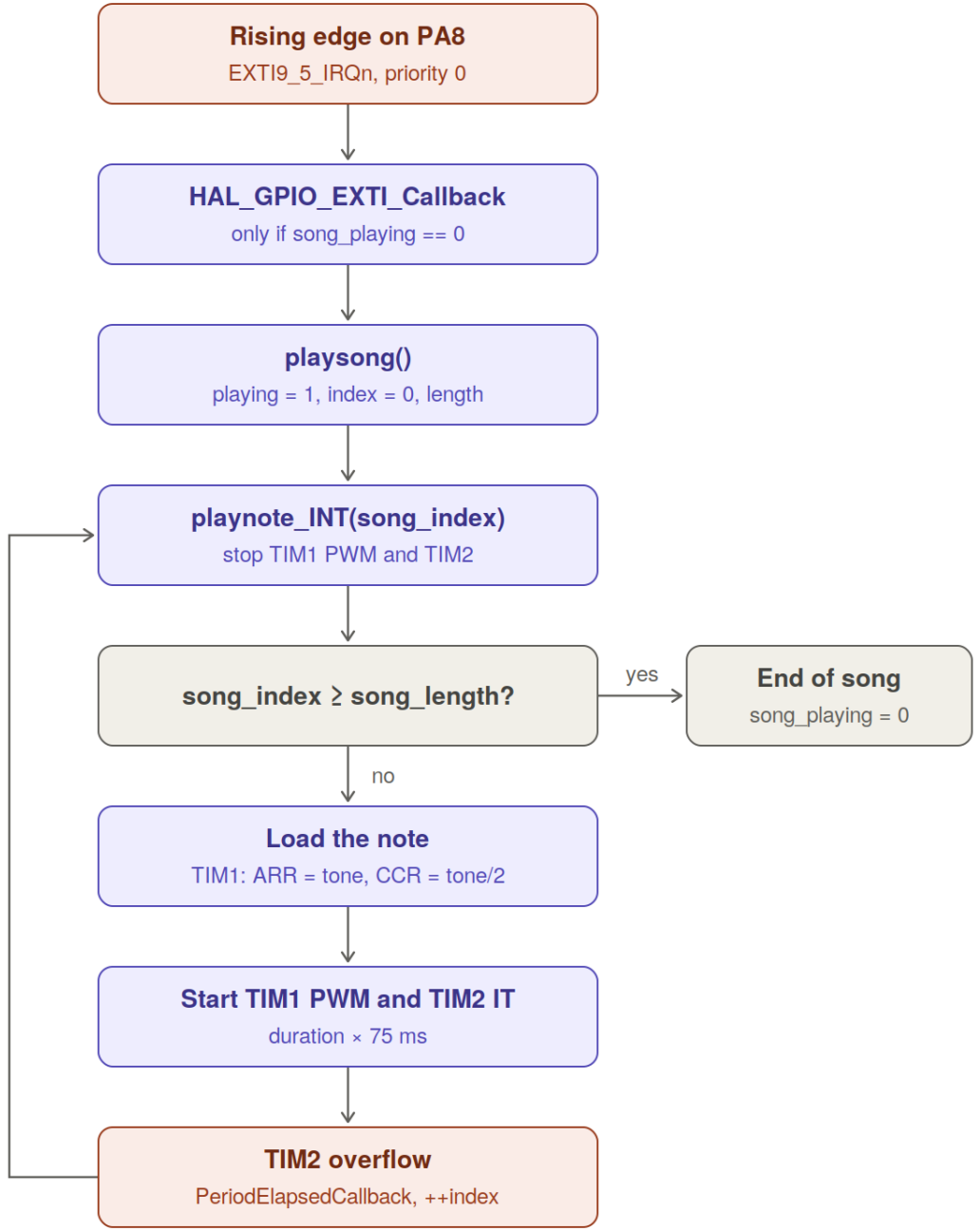
Each single increment of the counter clock lasts exactly 0,1 ms.

We need to multiply it by 10 to convert it in 1 ms.

So, to obtain the value of the ARR for a specific note we need to do:

$$Note\ duration\ in\ tempo\ * TEMPO\ * 10 = 3\ * 75\ ms\ * 10\ \frac{ticks}{ms}$$

HOMEWORK 3


Rising edge on PA8
EXTI9_5_IRQn, priority 0
HAL_GPIO_EXTI_Callback
only if song_playing == 0
playsong()
playing = 1, index = 0, length
playnote_INT(song_index)
stop TIM1 PWM and TIM2
song_index ≥ song_length?
yes
End of song
song_playing = 0
no
Load the note
TIM1: ARR = tone, CCR = tone/2
Start TIM1 PWM and TIM2 IT
duration × 75 ms
TIM2 overflow
PeriodElapsedCallback, ++index
hardware interrupt
user functions



POLITECNICO MILANO 1863 | DEPARTMENT OF ELECTRONICS INFORMATION AND BIOENGINEERING

Arianna.devecchi@polimi.it
Federica.villa@polimi.it
www.polimi.it

POLITECNICO MILANO 1863 | DIPARTIMENTO DI ELETTRONICA INFORMAZIONE E BIOINGEGNERIA